

Cerebras-Native Compute Software Platform

Research on a CUDA / ROCm-class ecosystem for Cerebras WSE and CS systems, including Mojo and Rust language frontends

Purpose: determine whether Cerebras can expose a general-purpose accelerator programming ecosystem comparable in strategic role to NVIDIA CUDA or AMD ROCm, and identify the most realistic architecture, implementation paths, developer interfaces, and migration strategy.

Research basis: public Cerebras documentation and publications available as of September 10, 2026, including the newly introduced CS-4 / WSE-3 Turbo generation, Cerebras SDK 1.4 documentation, Cerebras training software documentation, CUDA/ROCm documentation, recent academic work on Cerebras compilation and CUDA-to-CSL translation, and current Mojo and Rust compiler documentation and project work on MLIR-oriented heterogeneous compilation. [21][22][23][24][25]

1. Executive conclusion

A Cerebras equivalent to CUDA/ROCm is technically feasible, but the correct target is not “CUDA for a different chip.” Cerebras needs a native, layered compute platform that exposes the distinctive WSE execution model while giving developers a familiar accelerator software contract: device discovery, memory, kernels, asynchronous execution, compilation, libraries, profiling, debugging, packaging, and framework integration.

The strongest architecture is a stack centered on a stable Cerebras Compute API (working name: CCA), a public hardware abstraction/runtime, a modern MLIR-based compiler pipeline, a low-level kernel DSL that preserves CSL-level control, and high-performance libraries. CUDA source compatibility should be a migration product, not the semantic foundation.

This direction is especially timely because the platform is moving from a single-wafer-centric product toward rack-scale CS-4 systems. CS-4 contains three WSE-3 Turbo processors and a programmable I/O subsystem supporting RoCEv2 RDMA and direct wafer links; that creates a natural opportunity for a unified device + interconnect programming model rather than a wafer-local SDK alone. [1][2]

Dimension	CUDA / ROCm model	Cerebras opportunity
Primary abstraction	Threads, blocks, warps/wavefronts, device memory	Spatial dataflow: PEs, local SRAM, fabric routes, tasks, streams, regions
Kernel model	Host launches GPU kernels	Host schedules executable graphs / regions; PE tasks and dataflow kernels
Memory	HBM/device/global/shared/register hierarchies	Distributed local SRAM + external MemoryX/system memory + explicit fabric movement
Compiler	NVCC/PTX or HIP/LLVM	CSL compiler today; future public MLIR/CCA pipeline with hardware dialect and WSE backend
Libraries	cuBLAS/cuDNN/NCCL; ROCm equivalents	Cerebras BLAS, NN, attention, sparse, FFT, collectives, tensor movement, graph primitives
Frameworks	PyTorch, TensorFlow, JAX, Triton	PyTorch/JAX/ONNX frontends plus Triton/MLIR DSLs and CCA custom-op ABI
Tooling	debugger, profiler, sanitizer, Nsight/rocpfprof ecosystem	Fabric + PE debugger, topology-aware profiler, memory/fabric analyzers, simulator
Portability	CUDA compatibility / HIP portability	Native WSE programming + CUDA/HIP translation with semantic gap handling

2. Why Cerebras requires a different programming model

Cerebras WSE is fundamentally different from a GPU. WSE-3 has about 900,000 compute cores and 44 GB of distributed on-wafer SRAM; Cerebras reports roughly 21 PB/s aggregate memory bandwidth. The compute elements are arranged as a 2D mesh with local memories and fabric routers. [3][4]

The programming model is explicitly dataflow-oriented. Cerebras describes 32-bit “wavelets” moving through the fabric; arrival of particular wavelets can trigger tasks on processing elements. Each PE owns a small local program and memory. [4][5]

That means a direct CUDA-style mapping is structurally wrong. CUDA assumes an execution hierarchy where the program logically launches a grid of thread blocks onto a hardware scheduler. Cerebras programming more closely resembles spatial compilation: place computation, route data, establish event/task dependencies, and compile an executable placement.

The implication is important: the Cerebras software stack should make the architecture easier to use without hiding the architecture. Its competitive advantage comes from making the spatial/dataflow machine programmable at scale.

3. What already exists in the Cerebras stack

Cerebras already has many of the ingredients of a CUDA/ROCm-class ecosystem, but they are presented as several product-specific layers rather than one broad accelerator platform.

Existing component	Current role	CUDA/ROCm analogue	Gap for a broad ecosystem
CSL	C-like language for low-level WSE programs	CUDA C++ / HIP kernel language	Needs a stable public kernel ABI, more ergonomic host/device contract, and broader language integrations
cslc / SDK compiler	Compiles CSL to WSE programs and artifacts	nvcc / clang + device toolchain	Public compiler architecture and IR boundaries are not as open/standardized as LLVM-based stacks
SdkRuntime	Loads, runs, launches functions, H2D/D2H copies	CUDA runtime / HIP runtime	Needs durable multi-device context, queues/events, allocation, peer/wafer links, graph execution API
SdkLayout	Defines code regions, routing, connections	CUDA launch topology + distributed runtime configuration	Can become the topology layer of a general device runtime
Autogen kernels	Generates specialized/fused kernels	compiler-generated kernels / template libraries	Need general custom-op and kernel-generation APIs beyond selected loss functions
Cerebras PyTorch	PyTorch integration and CSX backend	PyTorch CUDA backend	Strong high-level entry point, but not yet an accelerator-neutral public systems API
Model Zoo	Reference models and utilities	framework ecosystem / NVIDIA NGC-like content	Need broader third-party library and community ecosystem
Fabric simulator	Local development/testing	CUDA device emulation is not equivalent; conceptually GPU simulators	Needs CI, deterministic tests, profiling, trace replay, and public benchmarking tooling

The current Cerebras SDK is already positioned as a general-purpose parallel-computing platform: its documentation exposes CSL, SdkRuntime, layout/routing, host-device transfers, and examples such as GEMV, GEMM, Cholesky, stencil, and sparse matrix-vector multiplication. [6][7]

Cerebras also has a history of an MLIR-based compiler stack. Its PyTorch integration previously described CIRH as MLIR-based and used graph rewrites, automatic kernel generation, and a placement stage before execution on the wafer. The company later moved its PyTorch integration toward Lazy Tensor Core. [8][9]

4. Current evidence that the opportunity is real

Three recent developments materially change the assessment.

- Cerebras has expanded beyond a single-processor model. CS-4 is built from three WSE-3 Turbo processors and a new programmable I/O subsystem. The I/O subsystem supports standards-based RoCEv2 RDMA and direct wafer links, with wafer-to-wafer latency as low as 2 microseconds according to Cerebras. [1][2]
- Cerebras has a mature enough low-level SDK for third parties to build non-neural-network workloads. The public SDK documentation includes dataflow kernels, routing, host/device streaming, layouts, and HPC examples; the company explicitly positions the SDK as a general-purpose parallel-computing platform. [6][10]
- The academic ecosystem is already moving toward compiler abstraction. A 2026 MLIR stencil compiler targets WSE2/WSE3 and reports performance comparable to manually optimized CSL for its stencil benchmarks. A separate 2026 paper, FABRICA, demonstrates automated CUDA-to-CSL translation with execution feedback and successful WSE3 execution, showing that CUDA migration can be systematized even though it is not a lexical translation problem. [11][12]

5. Proposed Cerebras Compute Platform (CCA)

Working name: Cerebras Compute Platform (CCA). This is a proposed architecture, not an existing Cerebras product name.

CCA should be defined as a platform contract rather than a single SDK package. The core design is seven layers: application/framework APIs; graph/runtime API; kernel API; compiler/IR; device runtime/driver; topology and communication; and libraries/tooling.

Layer	Proposed interface	Primary responsibility
L7 Applications	PyTorch, JAX, ONNX, Python/C++ APIs	Model and application authoring
L6 High-level compute API	Tensor ops, graphs, custom ops, collective APIs	Portable accelerator programming without topology details
L5 Kernel layer	CSL/CCA-Kernel, optional C++/Python DSL frontends	Author fused spatial/dataflow kernels
L4 Compiler IR	MLIR-based CCA dialects: Tensor, Spatial, Fabric, PE, WSE	Lower graph semantics into placement + communication + device code
L3 Runtime	Contexts, queues, events, memory, launch, executable graphs	Control execution and asynchronous data movement
L2 Device / fabric HAL	PE, SRAM banks, fabric links, I/O, wafer links, metrics	Stable hardware contract across WSE generations
L1 Driver / firmware	PCIe/network/control-plane, firmware, secure boot, telemetry	Actual system ownership and hardware lifecycle

6. What a CUDA/ROCm-like API should look like on Cerebras

The public API should be familiar in shape but native in semantics. A developer should be able to write code that feels like an accelerator runtime without pretending the accelerator contains GPU-style blocks and global memory.

CUDA concept	CCA concept	Semantics
cudaGetDevice	ccaDeviceEnumerate / ccaDeviceOpen	Enumerate CS systems, WSEs, and logical execution partitions
cudaMalloc	ccaAllocLocal / ccaAllocDistributed	Allocate local or distributed tensor/storage objects with placement metadata
cudaMemcpyAsync	ccaTransferAsync	H2D/D2H/wafer-to-wafer transfer; can select copy or streaming semantics
cudaStream_t	ccaQueue / ccaGraphQueue	Ordered asynchronous command stream or graph submission context
cudaEvent_t	ccaEvent	Completion/dependency token for asynchronous execution
kernel<<<grid,block>>>	ccaLaunch / ccaGraphSubmit	Launch compiled graph, spatial region, or

		host-callable dataflow kernel
shared memory	PE-local SRAM / scratch region	Explicit local storage with compiler-managed placement
cooperative groups	ccaCollective	Wafer/rack group collectives over fabric or wafer links
peer access	ccaPeer / ccaWaferLink	Explicit inter-WSE transfer and synchronization
NVRTC	ccaJIT / ccaCompile	Optional runtime specialization for shapes/topologies/versions
cudaGraph	ccaGraph	Precompiled execution graph with dataflow dependencies
CUDA profiler APIs	ccaTrace / ccaCounters	Hardware counters, fabric traffic, PE occupancy, routing and event traces

7. The native kernel model: do not clone CUDA threads

A key architectural decision is to expose “spatial kernels” rather than emulate the CUDA thread hierarchy. The unit of programming should be a spatial region composed of PEs, local state, explicit routes, and tasks. A kernel can declare inputs/outputs and a topology contract; the compiler maps that contract to physical coordinates.

A conceptual CCA kernel might look like this:

```
kernel gemm_tile(A, B, C) {
  tile t = placement::tile(/* shape */);
  local a = t.sram.load(A);
  local b = t.sram.load(B);
  task compute() when(a.ready && b.ready) {
    C.local += mma(a, b);
    fabric.send(C.local, direction::east);
  }
}
```

The point is not the exact syntax. The important abstractions are: tensor regions, placement constraints, local memory, asynchronous tasks, communication edges, collective operations, and compiler-managed topology.

8. Compiler strategy: MLIR should be the spine

The most defensible compiler strategy is to make MLIR the public extensibility spine while retaining Cerebras-specific lowering where proprietary optimization knowledge is essential. MLIR is specifically designed to connect high-level compiler representations to domain-specific hardware dialects and to reduce fragmentation across compiler stacks. [13]

A practical pipeline is:

- Framework IR (PyTorch FX / torch.export, StableHLO, ONNX, or CCA API graph).
- High-level optimization (fusion, constant folding, layout propagation, quantization, sparsity, shape specialization).
- Tensor / operator IR with explicit tiling and distribution metadata.
- Spatial IR: map tensors and operators onto PE regions, expose communication and locality.
- Fabric IR: route wavelets/data streams, allocate channels/colors, schedule dependencies.
- PE IR: lower compute to WSE instruction set / microthread model and local memory operations.
- Binary / executable package: per-region binaries + routing tables + metadata + launch manifest.

This is where Cerebras has an opportunity to leapfrog a conventional GPU SDK: instead of making developers manually express every routing decision, the compiler can infer routes from declarative placement constraints, then retain an escape hatch for experts.

9. Language strategy: Mojo and Rust as first-class frontends

A CUDA/ROCm-class Cerebras platform should not make Python and C/C++ the only programming entry points. Python remains essential for model development and ecosystem compatibility, while C and C++ remain important for ABI, operating-system, firmware and legacy interoperability. However, two newer systems languages are particularly valuable for a modern accelerator stack: Mojo and Rust. They address different parts of the problem and can share the same compiler/runtime contract.

9.1 Mojo: a natural MLIR-native accelerator language

Mojo is explicitly designed for high-performance AI infrastructure and heterogeneous hardware. Modular states that Mojo is built from the ground up using MLIR, exposes MLIR directly to developers, and lowers through MLIR/LLVM infrastructure. Mojo can therefore serve as a high-level but systems-oriented frontend for a Cerebras Spatial IR, rather than requiring Cerebras developers to learn every low-level detail of CSL. [21][22][23]

The architectural fit is strong because MLIR is designed around progressive lowering through dialects. A Cerebras Mojo integration could define a public or semi-public Cerebras dialect family for tensors, placement, spatial regions, fabric communication, collectives and device synchronization. Mojo source could lower into these dialects, after which Cerebras-specific passes perform placement, routing, scheduling and PE-level lowering. This creates a language path that resembles the design philosophy of a modern accelerator DSL while retaining a complete general-purpose language.

Mojo also has an important ecosystem property: its syntax and Python interoperability reduce the migration cost for AI developers while its static/compile-time features, parameterization and systems-programming model are intended for performance-critical code. This makes Mojo a plausible “native kernel language” for CCA, especially for researchers who want to move below Python without adopting the full complexity of C++.

9.2 Rust: systems language for the host, runtime, security and potentially kernels

Rust is equally important, but for a different reason. Standard rustc currently lowers Rust through HIR/THIR/MIR and then normally lowers MIR to LLVM IR; MIR is Rust's own mid-level representation used for borrow checking, analysis, optimization and code generation. Rust does not currently use MLIR as its standard compilation IR. [24][25]

That distinction is not a weakness for the Cerebras proposal. Rust has a backend abstraction in rustc, with LLVM, Cranelift and GCC backends already supported, and rustc supports a configurable/custom codegen backend model. In 2026, the Rust project is explicitly exploring an MLIR-based backend for accelerator offload, including splitting compilation between an MLIR accelerated portion and the existing LLVM host backend. This is directly aligned with the architecture needed for Cerebras. [26][27][28]

The recommended Rust strategy is therefore two-phase. First, make Rust the preferred systems language for the Cerebras host runtime, driver bindings, package manager, compiler tooling, simulator infrastructure, orchestration, networking, security-sensitive services and developer tools. Second, develop an experimental rustc-to-Cerebras path in which a restricted or annotated subset of Rust is lowered into Cerebras Spatial IR rather than forcing spatial programs through ordinary LLVM machine-code lowering.

This could eventually look like rustc → MIR → Cerebras backend → Spatial/Fabric IR → WSE executable. Where LLVM semantics are useful, Rust can still use the existing MIR → LLVM route for the host CPU side. The important design principle is that LLVM IR should not be treated as the universal accelerator IR: it is excellent for conventional instruction-oriented targets, while the Cerebras spatial model needs richer representation of placement, data movement, topology and async task dependencies.

9.3 Mojo and Rust are complementary, not competing choices

A strong CCA ecosystem can use both languages without duplicating the platform. Mojo can be the “accelerator-native language” centered on MLIR and high-performance kernels; Rust can be the “systems-native language” centered on memory safety, deterministic services, compiler/tooling infrastructure and secure runtime components. Both compile against a shared CCA ABI, device runtime and Spatial IR contract.

The language architecture should therefore be “many frontends, one accelerator contract”: Python for high-level framework work; C/C++ for compatibility and low-level integration; Mojo for MLIR-native kernels and accelerator metaprogramming; Rust for safe systems programming and an eventual native kernel/backend path. The CCA runtime, binary format, library ABI and Spatial/Fabric IR remain language-neutral.

Dimension	Python	C/C++	Mojo	Rust
Primary role	Model/application orchestration	ABI, low-level legacy, firmware	Native AI/accelerator programming	Runtime, tooling, security, systems
Compiler path	Framework/compiler specific	LLVM/other compilers	MLIR → LLVM / target backends	MIR → LLVM normally; emerging MLIR backend work
Cerebras kernel fit	Indirect	High control, high complexity	Excellent candidate	Promising via dedicated backend
Memory safety	Dynamic / runtime dependent	Manual	Static + systems-oriented model	Strong compile-time safety
Spatial IR access	Framework-mediated	Compiler-dependent	Direct MLIR access	Via backend / FFI / future MLIR path

10. CUDA, HIP and Triton compatibility: what is realistic

There are three very different goals: API compatibility, source compatibility, and semantic compatibility. Cerebras should target them in that order.

Approach	Feasibility	Value	Recommendation
CUDA runtime API emulation	Low to medium	Lets existing host software discover/launch a Cerebras device	Implement only a deliberately scoped CCA compatibility shim, not full CUDA
CUDA C++ source translation	Medium for kernels, low for full apps	Reduces porting effort for isolated kernels	Use compiler/LLM-assisted CUDA-to-CCA/CSL translation; expect manual tuning
HIP-like portability layer	Medium	Familiar portability model across accelerators	Use as a secondary source frontend if industry demand materializes
Triton-like DSL backend	High and strategically strong	Lets one kernel language target multiple accelerator backends	Build a CCA Triton backend that lowers to Spatial/Fabric IR, rather than trying to execute CUDA PTX semantics
ONNX / StableHLO / OpenXLA frontend	High for graphs	Broad framework interoperability	Prioritize graph-level integration before low-level API emulation
SPIR-V/OpenCL execution model	Low	Standardized portable kernels	Poor semantic fit for explicit spatial dataflow and fabric programming

The 2026 FABRICA results are particularly instructive: the authors explicitly state that CUDA-to-CSL porting requires architectural remapping rather than syntax substitution, and demonstrate that automated translation improves materially when target knowledge and execution feedback are available. [12]

11. The real CUDA moat: libraries, not just kernels

The software ecosystem will not become CUDA-class merely by publishing a compiler. CUDA became strategically indispensable because optimized libraries absorb years of algorithm engineering and make high performance the default behavior. ROCm follows the same pattern with libraries, tools, compilers and runtimes. [14][15]

CCA library family	Examples	Why it matters
Linear algebra	ccaBLAS, GEMM/GEMV, batched matmul	Foundation for HPC and ML
Deep learning primitives	attention, normalization, activations, convolution, embedding	Makes frameworks fast by default
Collectives	all-reduce, all-gather, reduce-scatter, broadcast	Essential for multi-wafer and rack-scale execution
Sparse	SpMV, sparse GEMM, graph kernels	Natural fit for WSE fine-grained sparsity/dataflow
Signal processing	FFT, convolution, spectral ops	Expands beyond LLM-only workloads
Data movement	tensor reshape, pack/unpack, DMA, RDMA, wafer links	Makes distributed execution predictable
Graph / runtime primitives	execution graphs, barriers, events, scheduling	Enables low-overhead serving and training
Quantization	FP formats, INT8/INT4, scaling, calibration	Required for inference ecosystem competitiveness

12. Developer tooling required for ecosystem adoption

- cca-inspect: device topology, WSE generation, available memory, I/O, links, firmware and runtime versions.
- cca-profile: PE utilization, fabric bandwidth, queue latency, task activation latency, hot routes, memory pressure and kernel timelines.
- cca-trace: deterministic event trace with host, WSE, and inter-wafer timelines.
- cca-debug / ccsdb integration: source-level CSL/CCA debugging with PE coordinates and wavelet/task inspection.
- cca-sim: deterministic fabric simulator with replayable traces and CI support.
- cca-test: correctness harness for kernels across WSE generations and simulator/hardware.
- cca-pack: versioned binary package format containing executable regions, routing, ABI metadata and supported architecture versions.

The topology must be first-class in tooling. A GPU profiler asks “which kernel and which SMs?” A Cerebras profiler must additionally answer “where is the work, which fabric paths carry it, which PEs are starving, and where are queue/task dependencies creating bubbles?”

13. CS-4 changes the runtime problem

CS-4 is a three-WSE system. Cerebras describes a new programmable I/O subsystem with RoCEv2 RDMA and direct wafer links, making rack-scale heterogeneous connectivity a first-class architectural feature. [1][2]

That suggests the runtime should not stop at “device memory + kernel launch.” It should offer a hierarchy: PE -> WSE -> CS system -> rack / cluster.

Level	Execution object	Typical operations
PE region	Spatial kernel region	task activation, local SRAM, neighbor sends
WSE	Wafer executable	tensor streams, barriers, collectives, global graph execution
CS-4 system	Multi-WSE graph	wafer-to-wafer pipeline, model partitioning, communication scheduling
Rack / cluster	Distributed CCA job	RDMA, scheduling, model replicas, service placement

The API should make locality explicit but should not force the application to manually manage it unless it wants to. The runtime can expose both “managed placement” and “expert placement” modes.

14. Language-aware compiler architecture

The compiler should explicitly distinguish the language frontend from the accelerator IR. This avoids coupling the Cerebras execution model to any one language. A reference architecture is:

Mojo → Mojo MLIR → CCA Spatial dialects → Fabric/PE dialects → WSE executable

Rust host → rustc MIR → LLVM IR → host binary

Rust accelerator subset → rustc MIR → dedicated Cerebras codegen backend → CCA Spatial dialects → WSE executable
C/C++ → LLVM IR / CCA bindings → CCA runtime and libraries

Python / PyTorch / JAX / ONNX / StableHLO → graph/tensor IR → CCA Spatial dialects → WSE executable

This separation also makes the public compiler boundary clearer. Developers can write language-specific abstractions, while the optimizer and hardware team can evolve placement, communication and scheduling algorithms in the shared CCA compiler without redesigning every frontend.

15. Implementation roadmap

Phase	Scope	Deliverables	Indicative difficulty
0. Contract definition	Freeze platform architecture and compatibility goals	CCA API spec, ABI/versioning policy, executable format, topology model	Medium
1. Runtime foundation	Expose a stable device/runtime HAL around existing Cerebras capabilities	device/context/queue/event/memory/transfer/launch APIs	Medium-high
2. Compiler spine	Introduce public MLIR boundaries around graph and spatial compilation	CCA dialects + lowering pipeline + optimizer interfaces	High
3. Kernel ecosystem	Make CSL first-class; add higher-level kernel DSLs	kernel ABI, custom ops, library packaging, test suite	High
4. Libraries	Accelerated math, NN, communication and data movement	ccaBLAS/NN/collectives/sparse/FFT	Very high
5. Frameworks	PyTorch/JAX/ONNX/Triton integration	first-class backends and custom-op tooling	High
6. Toolchain	Profiler/debugger/simulator/perf DB	production-grade developer experience	High
7. Migration	CUDA/HIP translation and assisted porting	CUDA-to-CCA/CSL translator, compatibility layer, benchmark suite	High
8. Rack-scale	CS-4 and successors	multi-WSE graph runtime, RDMA/wafer-link APIs, scheduler integration	Very high

16. Recommended strategy

The strongest strategic position is “native programming platform first, compatibility second.” A Cerebras software ecosystem should deliberately resemble CUDA/ROCm at the product level while being architecturally different underneath.

- Publish a stable accelerator runtime API and ABI. Make device/context/queue/event/memory/graph semantics durable across hardware generations.
- Keep CSL as the expert escape hatch, but wrap it in a stable kernel ABI and library model so that developers are not forced to write low-level routing code for routine cases.
- Adopt MLIR as the public compiler integration boundary. Preserve proprietary hardware-specific optimization passes behind stable dialect/ABI boundaries where needed.

- Build a “Spatial IR” that models PE placement, distributed SRAM, dataflow tasks and fabric communication explicitly.
- Build libraries before chasing broad CUDA source compatibility. A fast GEMM/attention/collective stack creates more ecosystem value than a superficial API shim.
- Make Mojo/MLIR, Rust, Triton, PyTorch, JAX and ONNX frontends strategic. These become the developer acquisition funnel; CCA/CSL becomes the performance control plane.
- Introduce a CUDA-to-CSL/CCA migration toolkit with correctness testing and auto-tuning. Treat this as an assisted migration product, consistent with the evidence from FABRICA rather than promising source-level equivalence.
- Treat CS-4 rack-scale I/O as part of the compute programming model from the beginning. Avoid designing a WSE-only runtime that later has to be retrofitted for multi-wafer execution.

17. Lessons from other accelerator ecosystems

The most useful comparator is not only CUDA. Other accelerator vendors show different ways to expose non-GPU architectures.

Ecosystem	Key lesson for Cerebras
NVIDIA CUDA	The moat is the full stack: compiler + runtime + libraries + tooling + community.
AMD ROCm/HIP	A portable C++ API plus open compilers/libraries lowers migration barriers; CUDA compatibility alone is insufficient.
Intel oneAPI/SYCL	A standardized high-level programming model can decouple developers from vendor hardware, while vendor-specific compilers preserve performance.
Graphcore Poplar	A graph-oriented framework can expose non-GPU architectures effectively while supporting mainstream ML frameworks.
Tenstorrent TT-Forge + TT-Metalium	The strongest modern pattern is a layered stack: high-level MLIR compiler, NN API, and low-level open SDK; custom hardware access remains available.
Academic Cerebras tooling	Domain-specific MLIR pipelines and CUDA-to-CSL automation demonstrate that a spatial compiler ecosystem can be built around WSE semantics.

18. Open-source and ecosystem posture

Public Cerebras repositories show an increasing open-source footprint: the Model Zoo is Apache-2.0 and the SDK examples are public. At the same time, the downloadable low-level Cerebras SDK is distributed as a roughly 3 GB Singularity image under a Cerebras EULA. [16][17]

That split suggests an important product decision. A CUDA/ROCm-class ecosystem needs a stable, inspectable developer contract even if some compiler optimizations or firmware remain proprietary. A reasonable model is: open API specifications, open examples, open frontend/compiler integrations, public IR definitions, and a supported binary runtime/driver where proprietary components remain necessary.

19. Major risks and mitigations

Risk	Why it matters	Mitigation
Semantic mismatch with GPU programming	CUDA assumptions about threads and memory do not map cleanly	Native spatial abstractions + assisted translation
Compiler complexity	Placement/routing/scheduling is much harder than standard accelerator codegen	MLIR multi-level IR, declarative constraints, strong simulator and autotuning
Library gap	Users will fall back to GPUs if common operations are slower or missing	Fund libraries as a first-class product with compatibility tests and performance SLOs

Binary/ABI churn	New WSE generations can break third-party kernels	Stable CCA ABI, capability negotiation, fat executable packages
Tooling opacity	Performance debugging is harder without topology visibility	First-class topology tracing, profiling and deterministic simulation
Rack-scale complexity	CS-4 adds inter-wafer and RDMA concerns	Hierarchical runtime model and communication API from day one
Ecosystem fragmentation	Multiple DSLs can create confusion	Use MLIR as the common compiler spine and keep one canonical low-level kernel ABI

20. What should be built first

For a 12-18 month program, the highest-value path is not to implement all CUDA APIs. It is to make Cerebras custom programming feel like a coherent platform.

- Stable language-neutral CCA runtime ABI with C/C++, Rust and Python bindings covering devices, contexts, memory, queues, events, async transfers, executable graphs and multi-WSE topology.
- Stable kernel ABI + package format that can carry CSL binaries and metadata across SDK releases.
- MLIR-based Spatial/Fabric IR with public specifications and extension points.
- First-party libraries: GEMM/GEMV, attention, reductions, collectives, tensor transforms, sparse kernels, FFT.
- Triton + PyTorch custom-op integration as the developer acquisition path.
- Profiler + simulator + benchmark suite with public canonical kernels and repeatable performance reports.
- CUDA-to-CCA/CSL translation and auto-tuning as an optional migration accelerator.

21. Bottom line

21.1 Recommended language positioning

For a long-lived Cerebras software ecosystem, Python should remain the top-level integration language, but it should not be the only language through which developers can reach the accelerator. Mojo provides the most direct conceptual match to an MLIR-native accelerator language, while Rust provides a compelling foundation for safe, high-performance runtime and infrastructure software and is now developing an MLIR-offload direction. The combination is substantially more modern and strategically differentiated than reproducing a Python + C/C++-only stack.

A practical priority is therefore: (1) Rust for CCA runtime, tooling, simulator, package manager, orchestration and security-sensitive infrastructure; (2) Mojo for CCA native kernels and compiler-facing accelerator programming; (3) C/C++ for ABI compatibility and legacy/native interop; and (4) Python for frameworks, notebooks and application-level work. Over time, the boundaries can converge around the same CCA Spatial IR and library ABI.

Cerebras already possesses much of the technical substrate for a CUDA/ROCm-class ecosystem: a low-level programming language, compiler, host runtime, topology/layout tools, simulation, model framework support, and an emerging multi-WSE system architecture. The main missing piece is a single stable platform abstraction that unifies these capabilities for both ML and general compute. The platform should also become explicitly multi-language: Python for high-level AI, C/C++ for compatibility, Mojo as an MLIR-native accelerator language, and Rust as the safe systems and runtime language, with an experimental path toward Rust accelerator code generation.

The best product would therefore be a “Cerebras Compute Platform” that plays the same ecosystem role as CUDA/ROCm, but exposes spatial dataflow as a first-class programming model. CUDA compatibility should be layered on top through translation and compiler assistance, while the native platform focuses on the properties that make WSE unique: enormous distributed SRAM, fine-grained compute placement, ultra-high-bandwidth on-wafer fabric, asynchronous dataflow, and low-latency scale-out.

The strategic result would be larger than a CUDA clone: it would turn Cerebras from a specialized accelerator with an SDK into a programmable compute platform with a stable developer contract, a compiler ecosystem, a library moat, and an extensible path from a single PE region to a rack-scale CS system.

References

- [1] Cerebras Systems. “Introducing Cerebras CS-4: The Fastest AI Just Got Faster.” Aug. 18, 2026. <https://www.cerebras.ai/blog/introducing-cerebras-cs-4>
- [2] Cerebras Systems. “Product - System: CS-4.” Accessed Sept. 9, 2026. <https://www.cerebras.ai/cs4>
- [3] Cerebras Systems. “Cerebras Systems Unveils World’s Fastest AI Chip with Whopping 4 Trillion Transistors.” Mar. 2024. <https://www.cerebras.ai/press-release/cerebras-announces-third-generation-wafer-scale-engine>
- [4] Cerebras Systems. “US DOE Achieves 88x Performance Speedup with Cerebras CS-2 Over H100 in Materials Modeling.” <https://www.cerebras.ai/blog/netl-achieves-88x-performance-on-materials-modeling>
- [5] Cerebras Systems. “Topic 3: Streaming Wavelet Data,” SDK Documentation 1.4.0. <https://cerebras-sdk-docs-140.netlify.app/csl/code-examples/tutorial-topic-03-streaming-wavelet-data>
- [6] Cerebras Systems. “Cerebras SDK Documentation 1.4.0.” <https://cerebras-sdk-docs-140.netlify.app/>
- [7] Cerebras Systems. “Host Runtime and Tensor Streaming,” SDK Documentation 1.4.0. <https://cerebras-sdk-docs-140.netlify.app/tensor-streaming>
- [8] Cerebras Systems. “Supporting PyTorch on the Cerebras Wafer-Scale Engine.” <https://www.cerebras.ai/blog/supporting-pytorch-on-the-cerebras-wafer-scale-engine>
- [9] Cerebras Systems. “Cerebras Software Release 2.0: 50% Faster Training, PyTorch 2.0 Support, Diffusion Transformers, and More.” <https://www.cerebras.ai/blog/cerebras-software-release-2.0-50-faster-training-pytorch-2.0-support-diffusion-transformers-and-more>
- [10] Cerebras Systems. “Supercharge Your HPC Research with the Cerebras SDK.” May 1, 2024. <https://www.cerebras.net/blog/supercharge-your-hpc-research-with-the-cerebras-sdk>
- [11] Stawinoga et al. “An MLIR Lowering Pipeline for Stencils at Wafer-Scale.” arXiv:2601.17754, 2026. <https://arxiv.org/abs/2601.17754>
- [12] Luo et al. “FABRICA: Agentic CUDA-to-CSL Translation and Optimization for Wafer-Scale Systems.” arXiv:2608.25124, 2026. <https://arxiv.org/abs/2608.25124>
- [13] LLVM Project. “MLIR: Multi-Level Intermediate Representation.” Accessed Sept. 9, 2026. <https://mlir.llvm.org/>
- [14] NVIDIA. “CUDA Toolkit Documentation.” Accessed Sept. 9, 2026. <https://docs.nvidia.com/cuda/>
- [15] AMD. “What is ROCm?” ROCm Documentation, 2026. <https://rocm.docs.amd.com/en/develop/what-is-rocm.html>
- [16] Cerebras Systems. “Cerebras / modelzoo,” GitHub. <https://github.com/Cerebras/modelzoo>
- [17] Cerebras Systems. “Installation and Setup,” SDK Documentation 1.4.0. <https://cerebras-sdk-docs-140.netlify.app/installation-guide>
- [18] Tenstorrent. “Software Stack Overview,” 2026. <https://docs.tenstorrent.com/software/index.html>
- [19] Graphcore. “Poplar SDK Overview.” <https://docs.graphcore.ai/projects/sdk-overview/en/latest/overview.html>
- [20] Intel. “oneAPI: A New Era of Heterogeneous Computing.” <https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html>
- [21] Modular. “Mojo Manual.” Accessed Sept. 10, 2026. <https://docs.modular.com/mojo/manual/>
- [22] Modular. “Mojo inline MLIR reference.” Accessed Sept. 10, 2026. <https://docs.modular.com/mojo/reference/mojo-inline-mlir/>
- [23] Modular. “Why Mojo.” Accessed Sept. 10, 2026. <https://docs.modular.com/stable/mojo/why-mojo>
- [24] Rust Project. “Overview of the compiler,” Rust Compiler Development Guide. Accessed Sept. 10, 2026. <https://rustc-dev-guide.rust-lang.org/overview.html>
- [25] Rust Project. “The MIR (Mid-level IR),” Rust Compiler Development Guide. Accessed Sept. 10, 2026. <https://rustc-dev-guide.rust-lang.org/mir/index.html>
- [26] Rust Project Goals 2026. “High-Level ML Optimizations: MLIR based backend for rustc.” Accessed Sept. 10, 2026. <https://goals.rust-lang.org/2026/high-level-ml.html>
- [27] Rust Project. “Backend Agnostic Codegen,” Rust Compiler Development Guide. Accessed Sept. 10, 2026. <https://rustc-dev-guide.rust-lang.org/backend/backend-agnostic.html>
- [28] Rust Project. “TargetOptions / custom codegen backend,” rustc documentation. Accessed Sept. 10, 2026. https://doc.rust-lang.org/stable/nightly-rustc/rustc_target/spec/struct.TargetOptions.html